

Capitolo 01 - Fondamenti

Caratteristiche di Java

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere le principali caratteristiche del linguaggio Java;
- conoscere il ruolo della JVM;
- comprendere il concetto di portabilità;
- riconoscere i vantaggi della gestione automatica della memoria;
- identificare gli ambiti di utilizzo più comuni di Java.

Introduzione

Java è un linguaggio di programmazione general purpose sviluppato da Sun Microsystems e rilasciato nel 1995.

Fin dalla sua introduzione, Java è stato progettato con alcuni obiettivi fondamentali:

- semplicità;
- portabilità;
- sicurezza;
- robustezza;
- indipendenza dalla piattaforma.

Grazie a queste caratteristiche è diventato uno dei linguaggi più utilizzati nello sviluppo software professionale.

Portabilità

Uno dei principi più noti di Java è:

Write Once, Run Anywhere

Il codice sorgente viene compilato in bytecode ed eseguito dalla Java Virtual Machine (JVM).

Schema semplificato:

Sorgente Java → Compilatore → Bytecode → JVM → Sistema Operativo

Vantaggi:

- stesso programma su Windows, Linux e macOS;
- maggiore compatibilità;
- distribuzione semplificata.

Gestione automatica della memoria

Java utilizza il Garbage Collector.

Quando un oggetto non è più utilizzato, la JVM può liberare automaticamente la memoria occupata.

Vantaggi:

- minori memory leak;
- maggiore produttività;
- codice più sicuro.

Robustezza

Java è stato progettato per ridurre numerosi errori tipici dei linguaggi più vicini all'hardware.

Caratteristiche:

- controllo dei tipi;
- gestione delle eccezioni;
- verifica del bytecode;
- assenza di puntatori espliciti.

Sicurezza

Java integra numerosi meccanismi di sicurezza:

- type checking;
- controllo degli accessi;
- class loader;
- API crittografiche.

Programmazione orientata agli oggetti

Java è un linguaggio orientato agli oggetti.

I concetti principali sono:

- classi;
- oggetti;
- incapsulamento;
- ereditarietà;
- polimorfismo;
- astrazione.

Tali argomenti saranno approfonditi nei capitoli successivi.

Ambiti di utilizzo

Java viene utilizzato per:

- applicazioni enterprise;
- backend web;
- microservizi;
- sistemi distribuiti;
- strumenti desktop;
- automazione;
- testing.

Riepilogo

Java è un linguaggio:

- portabile;
- sicuro;
- robusto;
- orientato agli oggetti;
- dotato di gestione automatica della memoria.

Queste caratteristiche ne spiegano il successo e la diffusione nel mondo professionale.

Domande di verifica

1. Cosa significa "Write Once, Run Anywhere"?
2. Qual è il ruolo della JVM?
3. Cos'è il Garbage Collector?
4. Quali vantaggi offre la portabilità?
5. Perché Java è considerato robusto?

Esercizi

Esercizio 1

Ricerca le principali piattaforme che supportano Java.

Esercizio 2

Descrivi il percorso di esecuzione di un programma Java dal file `.java` fino all'esecuzione sulla JVM.

Esercizio 3

Spiega con parole tue il ruolo del Garbage Collector.

Operatori ed Espressioni

Obiettivi

Al termine di questo capitolo sarai in grado di:

- distinguere operatori e operandi;
- scrivere espressioni aritmetiche e logiche;
- utilizzare gli operatori di assegnazione;
- comprendere precedenza e associatività;
- utilizzare l'operatore ternario.

Introduzione

Gli operatori permettono di eseguire operazioni sui dati.

Un'operazione coinvolge uno o più operandi e produce un risultato.

Esempio:

5 + 3

- 5 e 3 sono gli operandi
- ◦ è l'operatore
- 8 è il risultato

L'intera costruzione costituisce un'espressione.

Espressioni

Un'espressione è una combinazione di valori, variabili e operatori che produce un risultato.

Esempi:

10

x

10 + 5

```
x > y
```

```
a && b
```

Operatori aritmetici

Operatore	Significato
+	addizione
-	sottrazione
*	moltiplicazione
/	divisione
%	resto

Esempio:

```
int a = 10;  
int b = 3;  
---
```

```
System.out.println(a + b);  
System.out.println(a - b);  
System.out.println(a * b);  
System.out.println(a / b);  
System.out.println(a % b);
```

Operatori di assegnazione

Operatore	Equivalente
=	assegnazione
+=	$x = x + \text{valore}$
-=	$x = x - \text{valore}$
*=	$x = x * \text{valore}$
/=	$x = x / \text{valore}$
%=	$x = x \% \text{valore}$

Esempio:

```
int x = 10;
```

```
---
```

```
x += 5;
```

```
x *= 2;
```

Incremento e decremento

```
x++;  
++x;  
x--;  
--x;
```

Esempio:

```
int x = 5;
```

```
System.out.println(x++);
```

```
System.out.println(x);
```

```
---
```

```
System.out.println(++x);
```

Operatori relazionali

Operatore	Significato
==	uguale
!=	diverso
>	maggiore
<	minore
>=	maggiore o uguale
<=	minore o uguale

Esempio:

```
int a = 10;  
int b = 5;  
---
```

```
System.out.println(a > b);  
System.out.println(a == b);
```

Operatori logici

Operatore	Significato		
&&	AND		
			OR
!	NOT		

Esempio:

```
boolean maggiorenne = true;  
boolean patente = true;  
  
boolean puoGuidare = maggiorenne && patente;
```

Operatore ternario

Sintassi:

```
condizione ? valoreSeVero : valoreSeFalso;
```


Esempio:

```
int max = (a > b) ? a : b;
```

Precedenza degli operatori

Esempio:

```
int risultato = 2 + 3 * 4;
```

Risultato:

14

perché la moltiplicazione ha precedenza sull'addizione.

Per modificare l'ordine si utilizzano le parentesi:

```
int risultato = (2 + 3) * 4;
```

Risultato:

20

Errori comuni

Divisione intera

```
System.out.println(5 / 2);
```

Risultato:

```
2
```

non:

```
2.5
```

Confronto tra numeri e assegnazione

Errato:

```
if(x = 5)
```

Corretto:

```
if(x == 5)
```

Domande di verifica

1. Qual è la differenza tra operatore e operando?
2. Cos'è un'espressione?
3. Quali operatori restituiscono un valore booleano?
4. Qual è la differenza tra `x++` e `++x`?
5. A cosa serve l'operatore ternario?

Esercizi

Esercizio 1

Calcolare area e perimetro di un rettangolo.

Esercizio 2

Determinare il maggiore tra due numeri usando il ternario.

Esercizio 3

Verificare se un numero è compreso tra 10 e 100.

Esercizio 4

Simulare un login verificando username e password mediante operatori logici.

Esercizio 5

Prevedere il risultato delle seguenti espressioni senza eseguire il programma.

Variabili e Costanti

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere il concetto di variabile;
- dichiarare e inizializzare variabili;
- utilizzare correttamente gli identificatori;
- distinguere tra variabili locali e attributi;
- definire costanti mediante il modificatore `final`;
- acquisire dati da tastiera.

Introduzione

Durante l'esecuzione di un programma è necessario memorizzare informazioni da elaborare.

In Java questi dati vengono conservati all'interno di variabili, ovvero aree di memoria identificate da un nome e associate a un tipo di dato.

Le variabili rappresentano uno degli elementi fondamentali della programmazione e consentono di memorizzare, modificare e utilizzare valori durante l'esecuzione del programma.

Cos'è una variabile

Una variabile è:

- un'area di memoria;
- identificata da un nome;
- associata a un tipo di dato;
- utilizzata per memorizzare un valore.

Esempio:

```
int eta;  
double stipendio;  
String nome;
```

In questo esempio sono state dichiarate tre variabili di tipo differente.

Dichiarazione delle variabili

Prima di poter utilizzare una variabile è necessario dichiararla.

Sintassi:

```
tipo nomeVariabile;
```

Esempi:

```
int base;  
int altezza;  
double area;
```

Una variabile può essere dichiarata una sola volta all'interno dello stesso ambito.

Assegnazione di un valore

L'assegnazione consente di memorizzare un valore all'interno di una variabile.

Sintassi:

```
nomeVariabile = valore;
```

Esempio:

```
int base;  
base = 5;
```

L'assegnazione può utilizzare anche il risultato di un'espressione.

```
area = base * altezza / 2;
```

Dichiarazione e inizializzazione

Spesso dichiarazione e assegnazione vengono effettuate contemporaneamente.

Esempio:

```
int base = 5;  
int altezza = 10;  
double area = base * altezza / 2;
```

Questa forma è generalmente preferibile perché riduce il rischio di utilizzare variabili non inizializzate.

Convenzioni per i nomi

I nomi delle variabili devono rispettare alcune regole:

- possono contenere lettere, numeri e underscore;
- non possono iniziare con un numero;
- non possono coincidere con parole chiave del linguaggio;
- distinguono tra maiuscole e minuscole.

Esempi validi:

```
eta  
numeroStudenti  
totaleImporto
```

Esempi non validi:

```
2numero  
class  
double
```

Variabili locali

Le variabili dichiarate all'interno di un metodo sono dette variabili locali.

Esistono solamente durante l'esecuzione del metodo.

Esempio:

```
public static void main(String[] args) {  
    int numero = 10;  
    System.out.println(numero);  
}
```

La variabile `numero` non è accessibile all'esterno del metodo.

Attributi di classe

Le variabili dichiarate all'interno di una classe ma all'esterno dei metodi sono chiamate attributi.

Esempio:

```
public class Studente {  
    String nome;  
    int eta;  
}
```

Gli attributi rappresentano lo stato degli oggetti e saranno approfonditi nei capitoli dedicati alla programmazione orientata agli oggetti.

Scope delle variabili

Lo scope rappresenta l'ambito di visibilità di una variabile.

Una variabile è utilizzabile solamente all'interno del blocco in cui è stata dichiarata.

Esempio:

```
if(true){  
    int x = 10;  
}
```

```
System.out.println(x); // errore
```

La variabile `x` non è visibile al di fuori del blocco `if` .

Costanti

Una costante è una variabile il cui valore non può essere modificato dopo l'inizializzazione.

In Java si utilizza il modificatore `final`.

Esempio:

```
final double IVA = 0.22;
```

Dopo l'assegnazione qualsiasi tentativo di modifica genera un errore di compilazione.

```
IVA = 0.25; // errore
```

Convenzioni per le costanti

Per convenzione le costanti vengono scritte in maiuscolo.

Esempi:

```
final double PI = 3.14159;  
final int MAX_STUDENTI = 30;  
final double IVA = 0.22;
```

Esempio completo

```
public class Triangolo {  
    public static void main(String[] args) {  
        int base = 5;  
        int altezza = 10;  
  
        double area = base * altezza / 2.0;  
  
        System.out.println("Area = " + area);  
    }  
}
```

Input da tastiera

Per acquisire dati inseriti dall'utente si utilizza frequentemente la classe `Scanner` .

Esempio:

```
import java.util.Scanner;

public class InputExample {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        System.out.print("Inserisci un numero: ");

        int numero = input.nextInt();

        System.out.println("Hai inserito: " + numero);

        ---

    }

}
```

Errori comuni

Utilizzare una variabile non inizializzata

```
int x;  
  
System.out.println(x);
```

Errore di compilazione.

Utilizzare un identificatore non valido

```
int 2numero = 10;
```

Errore di compilazione.

Tentare di modificare una costante

```
final double PI = 3.14;
```

```
PI = 3.14159;
```

Errore di compilazione.

Domande di verifica

1. Cos'è una variabile?
2. Qual è la differenza tra dichiarazione e assegnazione?
3. Cosa si intende per scope?
4. Quando si utilizza il modificatore `final` ?
5. Qual è la differenza tra una variabile locale e un attributo?

Esercizi

Esercizio 1

Dichiarare tre variabili per memorizzare nome, cognome ed età di una persona.

Esercizio 2

Calcolare l'area di un rettangolo utilizzando due variabili.

Esercizio 3

Definire una costante contenente l'aliquota IVA e utilizzarla per calcolare il prezzo finale di un prodotto.

Esercizio 4

Leggere da tastiera il nome e l'età di uno studente e visualizzarli a video.

Esercizio 5

Scrivere un programma che scambi il contenuto di due variabili utilizzando una terza variabile temporanea.

Tipi Primitivi

Obiettivi

Al termine di questo capitolo sarai in grado di:

- distinguere tipi primitivi e tipi reference;
- conoscere gli otto tipi primitivi di Java;
- scegliere il tipo più appropriato per rappresentare un dato;
- dichiarare e inizializzare variabili primitive;
- comprendere il concetto di precisione e intervallo di valori.

Introduzione

Ogni dato memorizzato in un programma possiede un tipo.

Il tipo determina:

- quali valori possono essere memorizzati;
- quanta memoria viene utilizzata;
- quali operazioni possono essere eseguite.

Java mette a disposizione due grandi categorie di tipi:

- tipi primitivi;
- tipi reference (oggetti).

In questo capitolo analizzeremo i tipi primitivi.

Tipi primitivi e tipi reference

I tipi primitivi rappresentano valori semplici e sono incorporati direttamente nel linguaggio.

Esempi:

```
int eta = 20;  
double prezzo = 12.5;  
boolean attivo = true;
```

I tipi reference rappresentano invece oggetti.

Esempi:

```
String nome = "Mario";  
Scanner input = new Scanner(System.in);
```

I tipi reference saranno approfonditi nei capitoli dedicati agli oggetti.

Gli otto tipi primitivi

Java definisce otto tipi primitivi:

Categoria	Tipo
Interi	byte
Interi	short
Interi	int
Interi	long
Reali	float
Reali	double
Caratteri	char
Logici	boolean

Tipi interi

I tipi interi consentono di rappresentare numeri senza parte decimale.

Tipo	Dimensione	Intervallo
byte	8 bit	-128 ÷ 127
short	16 bit	-32.768 ÷ 32.767
int	32 bit	±2 miliardi
long	64 bit	±9 quintilioni

Esempio:

```
byte livello = 10;  
short anno = 2025;  
int popolazione = 590000000;  
long distanza = 3844000000L;
```

Il tipo int

Il tipo più utilizzato per i numeri interi è `int`.

Esempio:

```
int studenti = 25;  
int punti = 100;  
int temperatura = -5;
```

Nella maggior parte dei programmi è il tipo consigliato per rappresentare valori interi.

Il tipo long

Quando i valori superano il limite di `int` si utilizza `long` .

Esempio:

```
long popolazioneMondiale = 81000000000L;
```

Il suffisso **L** indica un letterale di tipo long.

Tipi reali

I tipi reali consentono di rappresentare numeri con parte decimale.

Tipo	Dimensione	Precisione
float	32 bit	circa 7 cifre
double	64 bit	circa 15 cifre

Esempio:

```
float temperatura = 23.5f;  
double piGreco = 3.141592653589793;
```

Il tipo double

Per i numeri decimali il tipo consigliato è generalmente `double`.

Esempio:

```
double prezzo = 19.99;  
double area = 123.456;
```

Offre una precisione maggiore rispetto a `float` .

Il tipo float

Il tipo `float` utilizza meno memoria ma offre una precisione inferiore.

Esempio:

```
float voto = 28.5f;
```

Il suffisso `f` è obbligatorio.

Il tipo char

Il tipo `char` rappresenta un singolo carattere Unicode.

Esempi:

```
char iniziale = 'M';  
char simbolo = '$';  
char lettera = 'A';
```

I caratteri sono racchiusi tra apici singoli.

Il tipo boolean

Il tipo `boolean` rappresenta valori logici.

Può assumere esclusivamente due valori:

```
true  
false
```

Esempio:

```
boolean maggiorenne = true;  
boolean autenticato = false;
```

I valori booleani sono utilizzati nelle condizioni e nelle strutture di controllo.

Dichiarazione e inizializzazione

Esempi di dichiarazione e inizializzazione:

```
byte b = 10;  
short s = 1000;  
int i = 50000;  
long l = 50000000000L;  
  
float f = 3.14f;  
double d = 3.14159265;  
  
char c = 'A';  
  
boolean ok = true;
```


Letterali

Un letterale è un valore scritto direttamente nel codice.

Esempi:

```
100
```

```
3.14
```

```
'A'
```

```
true
```

Ogni letterale possiede un tipo associato.

Valori di default degli attributi

Gli attributi di una classe ricevono automaticamente un valore iniziale.

Tipo	Valore di default
byte	0
short	0
int	0
long	0
float	0.0
double	0.0
char	'\u0000'
boolean	false

Le variabili locali, invece, devono sempre essere inizializzate prima dell'utilizzo.

Esempio completo

```
public class TipiPrimitivi {  
    public static void main(String[] args) {  
        int eta = 25;  
        double altezza = 1.82;  
        char sesso = 'M';  
        boolean studente = true;  
  
        System.out.println(eta);  
        System.out.println(altezza);  
        System.out.println(sesso);  
        System.out.println(studente);  
        ---  
    }  
}
```

Errori comuni

Dimenticare il suffisso L

Errato:

```
long valore = 50000000000;
```

Corretto:

```
long valore = 50000000000L;
```

Dimenticare il suffisso f

Errato:

```
float prezzo = 12.5;
```

Corretto:

```
float prezzo = 12.5f;
```


Utilizzare un char con doppi apici

Errato:

```
char lettera = "A";
```

Corretto:

```
char lettera = 'A';
```

Domande di verifica

1. Quali sono gli otto tipi primitivi di Java?
2. Qual è la differenza tra `int` e `long` ?
3. Quando è preferibile utilizzare `double` ?
4. Quali valori può assumere un `boolean` ?
5. Perché un `char` utilizza gli apici singoli?

Esercizi

Esercizio 1

Dichiarare una variabile per ciascuno degli otto tipi primitivi.

Esercizio 2

Memorizzare nome, età, altezza e stato di iscrizione di uno studente utilizzando i tipi più appropriati.

Esercizio 3

Calcolare area e circonferenza di un cerchio utilizzando variabili di tipo `double` .

Esercizio 4

Scrivere un programma che stampi il valore massimo rappresentabile da `byte`, `short`, `int` e `long`.

Esercizio 5

Creare una tabella che associ a ogni tipo primitivo dimensione, intervallo e utilizzo tipico.

Control Flow

Obiettivi

Al termine di questo capitolo sarai in grado di:

- controllare il flusso di esecuzione di un programma;
- utilizzare istruzioni condizionali;
- utilizzare cicli determinati e indeterminati;
- interrompere o modificare il comportamento di un ciclo;
- scegliere la struttura di controllo più adatta a un problema.

Introduzione

Per impostazione predefinita le istruzioni di un programma vengono eseguite in sequenza, dall'alto verso il basso.

Spesso però è necessario:

- eseguire istruzioni solo in determinate condizioni;
- ripetere più volte un blocco di codice;
- scegliere tra percorsi alternativi.

Per questo Java mette a disposizione diverse strutture di controllo del flusso.

Le strutture di controllo

Le principali strutture di controllo sono:

- `if`
- `if-else`
- `switch`
- `while`
- `do-while`
- `for`
- `for-each`

Esse consentono di implementare qualsiasi algoritmo.

Selezione Condizionale

L'istruzione if

L'istruzione `if` esegue un blocco di codice solamente se una condizione risulta vera.

Sintassi:

```
if (condizione) {  
    // istruzioni  
}
```

Esempio:

```
int eta = 20;  
---  
  
if (eta >= 18) {  
    System.out.println("Maggiorenne");  
}
```

if - else

Permette di gestire due casi alternativi.

Sintassi:

```
if (condizione) {  
    // caso vero  
}  
else {  
    // caso falso  
}
```

Esempio:

```
int voto = 22;  
  
if (voto >= 18) {  
    System.out.println("Promosso");  
} else {  
    System.out.println("Respinto");  
}
```


if - else if - else

Permette di gestire più condizioni.

Esempio:

```
int voto = 27;

if (voto >= 28) {
    System.out.println("Ottimo");
}
else if (voto >= 24) {
    System.out.println("Buono");
}
else if (voto >= 18) {
    System.out.println("Sufficiente");
}
else {
    System.out.println("Insufficiente");
}
```

Selezione Multipla

L'istruzione switch

Quando occorre scegliere tra molte alternative, `switch` può risultare più leggibile di una lunga sequenza di `if`.

Sintassi:

```
switch (espressione) {  
    case valore1:  
        break;  
  
    case valore2:  
        break;  
  
    default:  
  
}
```


Esempio:

```
int giorno = 3;

switch (giorno) {

    case 1:
        System.out.println("Lunedì");
        break;

    case 2:
        System.out.println("Martedì");
        break;

    ---

    case 3:
        System.out.println("Mercoledì");
        break;

    default:
        System.out.println("Valore non valido");
}
```

Iterazione

Il ciclo while

Il ciclo `while` ripete un blocco di istruzioni finché una condizione rimane vera.

Sintassi:

```
while (condizione) {  
    // istruzioni  
}
```


Esempio:

```
int contatore = 1;  
---  
while (contatore <= 5) {  
    System.out.println(contatore);  
    contatore++;  
}
```

Il ciclo do-while

Il ciclo `do-while` garantisce almeno una esecuzione del blocco.

Sintassi:

```
do {  
    // istruzioni  
} while (condizione);
```

Esempio:

```
int numero = 1;  
---  
  
do {  
    System.out.println(numero);  
    numero++;  
} while (numero <= 5);
```

Il ciclo for

Il ciclo `for` è indicato quando il numero di iterazioni è noto.

Sintassi:

```
for (inizializzazione; condizione; aggiornamento) {  
    // istruzioni  
}
```

Esempio:

```
for (int i = 1; i <= 10; i++) {  
    ---  
    System.out.println(i);  
}
```

Il ciclo for-each

Permette di attraversare tutti gli elementi di una collezione o di un array.

Esempio:

```
int[] numeri = {10, 20, 30, 40};  
  
for (int numero : numeri) {  
    System.out.println(numero);  
}
```

Gli array verranno approfonditi nel capitolo successivo.

Break e Continue

break

L'istruzione `break` interrompe immediatamente il ciclo.

Esempio:

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) {  
        break;  
    }  
    System.out.println(i);  
}
```

Output:

```
1  
2  
3  
4
```

continue

L'istruzione `continue` interrompe l'iterazione corrente e passa alla successiva.

Esempio:

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) {  
        continue;  
    }  
    System.out.println(i);  
}
```

Output:

```
1  
2  
3  
4  
6  
7  
8  
9  
10
```

Pattern Fondamentali

Ricerca

Verificare se un valore è presente in un insieme di dati.

Esempio:

```
boolean trovato = false;  
  
for (int i = 0; i < valori.length; i++) {  
    if (valori[i] < 0) {  
        trovato = true;  
        break;  
    }  
}
```

Filtro

Elaborare solo gli elementi che soddisfano una condizione.

Esempio:

```
for (int i = 1; i <= 100; i++) {  
    if (i % 2 == 0) {  
        System.out.println(i);  
    }  
}
```

Accumulatore

Memorizzare progressivamente un risultato.

Esempio:

```
int somma = 0;
for (int i = 1; i <= 100; i++) {
    somma += i;
}
```

Contatore

Contare quante volte si verifica una condizione.

Esempio:

```
int pari = 0;

for (int i = 1; i <= 100; i++) {

    if (i % 2 == 0) {
        pari++;
    }
}
```

Cicli annidati

Un ciclo può essere contenuto all'interno di un altro ciclo.

Esempio:

```
for (int r = 0; r < 5; r++) {  
    for (int c = 0; c < 5; c++) {  
        System.out.print("*");  
    }  
    System.out.println();  
}
```

Output:

```
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *
```

Errori Comuni

DimENTICARE l'aggiornamento del contatore

```
while (x < 10) {  
    System.out.println(x);  
}
```

Provoca un ciclo infinito.

Utilizzare un punto e virgola dopo if

Errato:

```
if (x > 0);  
{  
    System.out.println(x);  
}
```

Utilizzare = invece di ==

Errato:

```
if (x = 10)
```

Corretto:

```
if (x == 10)
```

Domande di Verifica

1. Quando utilizzare un `if` ?
2. Qual è la differenza tra `while` e `do-while` ?
3. Quando è preferibile utilizzare un ciclo `for` ?
4. Qual è lo scopo dell'istruzione `break` ?
5. Qual è la differenza tra `break` e `continue` ?
6. Cosa si intende per ciclo annidato?
7. Cos'è un accumulatore?

Esercizi

Esercizio 1

Stampare i numeri da 1 a 100 utilizzando un ciclo `for` .

Esercizio 2

Stampare tutti i numeri pari compresi tra 1 e 100.

Esercizio 3

Calcolare la somma dei primi 100 numeri naturali.

Esercizio 4

Leggere numeri da tastiera finché l'utente inserisce un valore negativo.

Esercizio 5

Determinare il massimo tra 10 numeri inseriti dall'utente.

Esercizio 6

Stampare la tabellina di un numero.

Esercizio 7

Disegnare un quadrato di asterischi utilizzando cicli annidati.

Esercizio 8

Contare quante vocali sono presenti in una stringa.

Esercizio 9

Verificare se un numero è primo.

Esercizio 10

Realizzare il classico gioco "Indovina il numero" utilizzando cicli e condizioni.

Array

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere cosa sono gli array;
- dichiarare, creare e inizializzare array;
- accedere agli elementi mediante indice;
- attraversare un array utilizzando i cicli;
- utilizzare array multidimensionali;
- conoscere le principali operazioni offerte dalla classe `Arrays` .

Introduzione

Molto spesso un programma deve gestire un insieme di valori dello stesso tipo.

Ad

esempio:

- i voti di una classe;
- le temperature registrate durante una settimana;
- i risultati di una partita;
- i prezzi di una serie di prodotti.

Utilizzare una variabile per ogni valore sarebbe poco pratico.

Per questo motivo Java mette a disposizione gli array.

Cos'è un Array

Un array è una struttura dati che contiene una sequenza ordinata di elementi dello stesso tipo.

Caratteristiche principali:

- contiene elementi omogenei;
- ogni elemento è identificato da un indice;
- gli elementi sono memorizzati in posizioni consecutive logiche;
- la dimensione è fissa.

Esempio:

```
int[] voti = {28, 30, 25, 27};
```

Gli Array sono Oggetti

In Java gli array sono oggetti.

Vengono creati nell'heap mediante l'operatore `new` .

Esempio:

```
int[] numeri = new int[10];
```

La variabile `numeri` contiene un riferimento all'oggetto array.

Dichiarazione di un Array

Sintassi:

```
tipo[] nomeArray;
```

Esempi:

```
int[] voti;  
double[] prezzi;  
String[] nomi;
```

È possibile trovare anche la sintassi:

```
int voti[];
```

ma la forma consigliata è:

```
int[] voti;
```

Creazione di un Array

Dopo la dichiarazione è necessario creare l'array.

Esempio:

```
int[] voti;  
voti = new int[10];
```

L'array contiene 10 elementi indicizzati da 0 a 9.

Dimensione dell'Array

La dimensione viene specificata durante la creazione.

```
int[] numeri = new int[100];
```

Una volta creato, l'array non può essere ridimensionato.

La sua lunghezza è disponibile tramite l'attributo `length`.

```
System.out.println(numeri.length);
```

Inizializzazione Automatica

Quando viene creato un array, tutti gli elementi ricevono un valore iniziale.

Tipo	Valore iniziale
byte, short, int, long	0
float, double	0.0
char	'\u0000'
boolean	false
reference	null

Esempio:

```
int[] valori = new int[5];
```

Contenuto iniziale:

```
[0, 0, 0, 0, 0]
```


Inizializzazione con Valori

È possibile inizializzare un array al momento della dichiarazione.

Esempio:

```
int[] numeriPrimi = {2, 3, 5, 7, 11, 13};
```

Oppure:

```
String[] giorni = {  
    "Lunedì",  
    "Martedì",  
    "Mercoledì"  
};
```

Accesso agli Elementi

Gli elementi vengono identificati mediante un indice.

Sintassi:

```
nomeArray[indice]
```

Esempio:

```
int[] voti = {28, 30, 25};  
  
System.out.println(voti[0]);  
System.out.println(voti[1]);  
System.out.println(voti[2]);
```

Modifica degli Elementi

Gli elementi possono essere modificati mediante assegnazione.

Esempio:

```
int[] voti = {28, 30, 25};  
voti[2] = 27;
```

Scorrere un Array con for

L'utilizzo più comune prevede un ciclo `for` .

Esempio:

```
int[] numeri = {10, 20, 30, 40};  
for (int i = 0; i < numeri.length; i++) {  
    System.out.println(numeri[i]);  
}
```


Scorrere un Array con for-each

Quando non è necessario conoscere l'indice, è possibile utilizzare il ciclo for-each.

Esempio:

```
int[] numeri = {10, 20, 30, 40};  
for (int numero : numeri) {  
    System.out.println(numero);  
}
```

Algoritmi Fondamentali sugli Array

Somma degli elementi

```
int somma = 0;

for (int valore : numeri) {
    somma += valore;
}
```

Ricerca di un elemento

```
boolean trovato = false;

for (int valore : numeri) {

    if (valore == 30) {
        trovato = true;
        break;
    }
}
```

Ricerca del massimo

```
int massimo = numeri[0];  
for (int valore : numeri) {  
    if (valore > massimo) {  
        massimo = valore;  
    }  
}
```

Conteggio

```
int pari = 0;

for (int valore : numeri) {

    if (valore % 2 == 0) {
        pari++;
    }
}
```

Array Multidimensionali

Un array multidimensionale è un array che contiene altri array.

La forma più comune è la matrice.

Creazione di una matrice

```
int[][] matrice = new int[3][4];
```

La matrice contiene:

- 3 righe;
- 4 colonne.

Accesso agli elementi

```
matrice[1][2] = 10;
```

Significa:

- riga 1
- colonna 2

Scorrimento di una matrice

```
for (int r = 0; r < matrice.length; r++) {  
    for (int c = 0; c < matrice[r].length; c++) {  
        System.out.print(matrice[r][c] + " ");  
    }  
    System.out.println();  
}
```

Array Irregolari

Le righe possono avere lunghezze differenti.

Esempio:

```
int[][] triangolo = {  
    {1, 2, 3, 4},  
    {1, 2, 3},  
    {1, 2},  
    {1}  
};
```

Array di Oggetti

Un array può contenere riferimenti a oggetti.

Esempio:

```
String[] nomi = {  
    "Mario",  
    "Anna",  
    "Luca"  
};
```

Oppure:

```
Studente[] studenti = new Studente[30];
```

Gli elementi dell'array contengono riferimenti agli oggetti.

La Classe Arrays

Il package `java.util` contiene la classe di utilità `Arrays` .

Importazione:

```
import java.util.Arrays;
```


Ordinamento

```
Arrays.sort(neri);
```

Confronto

```
Arrays.equals(a, b);
```

Riempimento

```
Arrays.fill(voti, 18);
```

Ricerca Binaria

```
Arrays.binarySearch(neri, 30);
```

L'array deve essere ordinato.

Errori Comuni

Accesso fuori dai limiti

Errato:

```
int[] numeri = new int[5];  
System.out.println(numeri[5]);
```

Genera:

```
ArrayIndexOutOfBoundsException
```

Utilizzare un array non inizializzato

```
int[] numeri;  
  
System.out.println(numeri.length);
```

Genera:

```
NullPointerException
```

Confondere length con length()

Errato:

```
numeri.length();
```

Corretto:

```
numeri.length
```


Domande di Verifica

1. Cos'è un array?
2. Perché si dice che gli array sono oggetti?
3. Qual è il primo indice di un array?
4. Qual è l'ultimo indice valido?
5. Cosa restituisce l'attributo `length` ?
6. Qual è la differenza tra `for` e `for-each` ?
7. Cos'è un array multidimensionale?
8. A cosa serve la classe `Arrays` ?

Esercizi

Esercizio 1

Creare un array di 10 numeri interi e stamparne il contenuto.

Esercizio 2

Calcolare la somma degli elementi di un array.

Esercizio 3

Calcolare media, minimo e massimo di un array.

Esercizio 4

Contare quanti elementi pari e dispari sono presenti.

Esercizio 5

Verificare se un valore è presente nell'array.

Esercizio 6

Invertire il contenuto di un array.

Esercizio 7

Ordinare un array utilizzando `Arrays.sort()` .

Esercizio 8

Realizzare la ricerca di un elemento mediante scansione lineare.

Esercizio 9

Creare e stampare una matrice 5x5.

Esercizio 10

Calcolare la somma degli elementi della diagonale principale di una matrice.

Stringhe

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere il ruolo della classe `String` ;
- creare e manipolare stringhe;
- utilizzare i principali metodi della classe `String` ;
- confrontare correttamente due stringhe;
- convertire valori tra stringhe e tipi primitivi;
- utilizzare `StringBuilder` per costruire stringhe in modo efficiente.

Introduzione

Le stringhe rappresentano sequenze di caratteri e sono utilizzate in quasi tutti i programmi.

Esempi di informazioni rappresentate tramite stringhe:

- nomi e cognomi;
- indirizzi email;
- messaggi;
- dati provenienti da file;
- dati inseriti dall'utente.

In Java le stringhe sono rappresentate dalla classe `String`.

La Classe String

Una stringa è un oggetto della classe `String`.

Esempio:

```
String nome = "Mario";
```

La variabile `nome` contiene un riferimento a un oggetto `String`.

Creazione di Stringhe

Utilizzando un letterale

```
String linguaggio = "Java";
```

È il metodo più comune.

Utilizzando il costruttore

```
String linguaggio = new String("Java");
```

Questa forma è generalmente meno utilizzata.

Stringhe e Caratteri

Una stringa è composta da una sequenza di caratteri.

Esempio:

```
String parola = "Java";
```

Indice	Carattere
0	J
1	a
2	v
3	a

Gli indici partono sempre da zero.

Lunghezza di una Stringa

Il metodo `length()` restituisce il numero di caratteri.

Esempio:

```
String s = "Java";  
System.out.println(s.length());
```

Output:

```
4
```


Accesso ai Caratteri

Il metodo `charAt ( )` restituisce il carattere presente in una determinata posizione.

Esempio:

```
String s = "Java";  
char c = s.charAt(2);  
System.out.println(c);
```

Output:

v

Concatenazione

Le stringhe possono essere concatenate mediante l'operatore `+`.

Esempio:

```
String nome = "Mario";  
String cognome = "Rossi";  
  
String completo = nome + " " + cognome;
```

Risultato:

```
Mario Rossi
```

Confronto tra Stringhe

Metodo equals()

Per confrontare il contenuto di due stringhe si utilizza `equals()` .

Esempio:

```
String a = "Java";  
String b = "Java";  
  
System.out.println(a.equals(b));
```

Output:

```
true
```

Metodo equalsIgnoreCase()

Ignora la differenza tra maiuscole e minuscole.

Esempio:

```
String a = "JAVA";  
String b = "java";  
  
System.out.println(a.equalsIgnoreCase(b));
```

Output:

```
true
```

Ricerca all'Interno di una Stringa

contains()

Verifica la presenza di una sottostringa.

```
String testo = "Java Programming";  
System.out.println(testo.contains("gram"));
```

Output:

```
true
```

indexOf()

Restituisce la posizione di un carattere o di una sottostringa.

```
String testo = "Java";  
System.out.println(testo.indexOf('v'));
```

Output:

```
2
```

Estrazione di Sottostringhe

Il metodo `substring()` consente di estrarre una porzione della stringa.

Esempio:

```
String linguaggio = "Java Programming";  
String parola = linguaggio.substring(0, 4);  
System.out.println(parola);
```

Output:

```
Java
```

Conversione Maiuscole e Minuscole

```
String s = "Java";
```

Maiuscolo:

```
s.toUpperCase();
```

Minuscolo:

```
s.toLowerCase();
```

Eliminazione degli Spazi

Il metodo `trim()` elimina gli spazi iniziali e finali.

Esempio:

```
String s = "  Java  ";  
System.out.println(s.trim());
```

Output:

```
Java
```

Sostituzione di Caratteri

```
String testo = "Java";  
String nuovoTesto = testo.replace('a', 'o');
```

Risultato:

Jovo

Divisione di una Stringa

Il metodo `split()` divide una stringa in più parti.

Esempio:

```
String csv = "Mario,Rossi,25";  
String[] dati = csv.split(",");
```

Risultato:

```
Mario  
Rossi  
25
```

Conversione Stringa → Numero

Intero

```
int numero = Integer.parseInt("123");
```

Double

```
double prezzo = Double.parseDouble("12.50");
```

Conversione Numero → Stringa

```
String s = String.valueOf(123);
```

Oppure:

```
String s = Integer.toString(123);
```


Stringhe e Array di Caratteri

Una stringa può essere convertita in un array di caratteri.

Esempio:

```
String parola = "Java";  
char[] caratteri = parola.toCharArray();
```

L'Immutabilità delle Stringhe

Le stringhe sono immutabili.

Una volta create, non possono essere modificate.

Esempio:

```
String s = "Java";  
  
s = s + " 21";
```

Non viene modificata la stringa originale.

Viene creato un nuovo oggetto `String`.

StringBuilder

Quando una stringa deve essere modificata molte volte è preferibile utilizzare `StringBuilder`.

Esempio:

```
StringBuilder sb = new StringBuilder();  
  
sb.append("Java");  
sb.append(" ");  
sb.append("Rocks");  
  
String risultato = sb.toString();
```

Output:

Java Rocks

Caratteri Speciali (Escape Sequences)

Sequenza	Significato
\n	nuova riga
\t	tabulazione
\	backslash
"	doppio apice
'	apice

Esempio:

```
System.out.println("Prima riga\nSeconda riga");
```


Text Blocks

I Text Blocks permettono di definire stringhe multilinea in modo leggibile.

Esempio:

```
String html = ""  
    <html>  
        <body>  
            <h1>Hello Java</h1>  
        </body>  
    </html>  
    "";
```

Esempio Completo

```
public class DemoStringhe {  
    public static void main(String[] args) {  
        String nome = "Mario";  
        String cognome = "Rossi";  
  
        String completo = nome + " " + cognome;  
  
        System.out.println(completo);  
        System.out.println(completo.length());  
    }  
}
```

Errori Comuni

Utilizzare == per confrontare stringhe

Errato:

```
if(nome == "Mario")
```

Corretto:

```
if(nome.equals("Mario"))
```

Accesso a un indice inesistente

Errato:

```
String s = "Java";  
s.charAt(10);
```

Genera:

```
StringIndexOutOfBoundsException
```

Dimenticare il controllo di null

```
String s = null;  
s.length();
```

Genera:

```
NullPointerException
```

Domande di Verifica

1. Cos'è una stringa?
2. Qual è la differenza tra `char` e `String` ?
3. Cosa restituisce `length()` ?
4. A cosa serve `substring()` ?
5. Perché si utilizza `equals()` invece di `==` ?
6. Cosa significa che una stringa è immutabile?
7. Quando conviene utilizzare `StringBuilder` ?

Esercizi

Esercizio 1

Leggere un nome e stamparne la lunghezza.

Esercizio 2

Contare il numero di vocali presenti in una stringa.

Esercizio 3

Invertire una stringa.

Esercizio 4

Verificare se una parola è palindroma.

Esercizio 5

Convertire una frase in maiuscolo.

Esercizio 6

Estrarre nome e cognome da una stringa nel formato:

```
Mario Rossi
```

Esercizio 7

Dividere una riga CSV utilizzando `split()` .

Esercizio 8

Contare quante volte compare una determinata lettera.

Esercizio 9

Costruire una stringa utilizzando `StringBuilder` .

Esercizio 10

Leggere una frase e visualizzare la parola più lunga.

Casting

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere il concetto di conversione tra tipi;
- distinguere tra conversione implicita ed esplicita;
- utilizzare correttamente il casting dei tipi primitivi;
- riconoscere i casi in cui si può verificare una perdita di informazione;
- comprendere il ruolo della promotion nelle espressioni.

Introduzione

Durante l'esecuzione di un programma può essere necessario convertire un valore da un tipo a un altro.

Ad

esempio:

- trasformare un `int` in un `double` ;
- convertire un `double` in un `int` ;
- utilizzare insieme variabili di tipo diverso nella stessa espressione.

Java mette a disposizione due meccanismi:

- promotion (conversione automatica);
- casting (conversione esplicita).

Conversione Implicita (Promotion)

La promotion avviene automaticamente quando il passaggio è considerato sicuro e non comporta perdita di informazioni.

Esempio:

```
int numero = 10;  
long valore = numero;
```

Il compilatore esegue automaticamente la conversione.

Tipi Compatibili

Le conversioni implicite più comuni sono:

```
byte → short → int → long → float → double
```


Esempio:

```
int eta = 25;  
double valore = eta;
```

Output:

```
25.0
```

Promotion nelle Espressioni

Quando un'espressione contiene operandi di tipo diverso, Java converte automaticamente i valori verso il tipo più grande.

Esempio:

```
int a = 10;  
double b = 3.5;  
  
double risultato = a + b;
```

Il valore di `a` viene convertito automaticamente in `double`.

Il Casting Esplicito

Quando la conversione può comportare perdita di informazione, è necessario utilizzare il cast.

Sintassi:

```
(tipo) espressione
```

Esempio:

```
double prezzo = 19.99;  
int intero = (int) prezzo;
```

Output:

19

La parte decimale viene eliminata.

Conversione da double a int

Esempio:

```
double pi = 3.14159;  
int valore = (int) pi;  
System.out.println(valore);
```

Output:

```
3
```


Il cast non arrotonda il valore.

Effettua semplicemente il troncamento della parte decimale.

Conversione da int a double

Esempio:

```
int numero = 10;  
double valore = numero;
```

In questo caso non è necessario alcun cast.

Conversione tra Tipi Interi

Esempio:

```
int numero = 120;  
byte b = (byte) numero;
```

Il cast è obbligatorio perché il tipo `byte` dispone di meno spazio.

Perdita di Informazione

Alcune conversioni possono alterare il valore originale.

Esempio:

```
int numero = 1000;  
byte b = (byte) numero;
```

Il valore memorizzato in `b` non sarà più 1000.

Esempio di Overflow

```
byte b = (byte) 261;  
  
System.out.println(b);
```

Output:

```
5
```

Il valore viene adattato alla capacità del tipo `byte` .

Casting e char

Il tipo `char` può essere convertito in un intero e viceversa.

Esempio:

```
char lettera = 'A';  
int codice = lettera;  
System.out.println(codice);
```

Output:

```
65
```

Conversione da int a char

```
int codice = 66;  
  
char lettera = (char) codice;  
  
System.out.println(lettera);
```

Output:

B

Ordine di Valutazione

Il punto in cui viene applicato il cast può modificare il risultato.

Esempio:

```
int a = (int) 3.0 * (int) 4.5;
```

Risultato:

12

Perché i valori vengono convertiti prima della moltiplicazione.

Esempio:

```
int b = (int) (3.0 * 4.5);
```

Risultato:

13

Perché il prodotto viene eseguito prima del cast.

Promotion dei Tipi Piccoli

Nelle espressioni aritmetiche Java promuove automaticamente:

```
byte  
short  
char
```

a tipo `int` .

Esempio:

```
byte a = 10;  
byte b = 20;  
---  
  
int risultato = a + b;
```

Il risultato dell'operazione è un `int`.

Conversione di Stringhe

Spesso è necessario convertire una stringa in un numero.

Esempio:

```
String testo = "123";  
int numero = Integer.parseInt(testo);
```

Oppure:

```
String testo = "3.14";  
---  
double valore = Double.parseDouble(testo);
```

Conversione di Numeri in Stringhe

```
int numero = 123;  
  
String testo = String.valueOf(numero);
```

Casi Non Consentiti

Il tipo `boolean` non può essere convertito in tipi numerici.

Errato:

```
boolean b = true;  
  
int x = (int) b;
```

Errore di compilazione.

Esempio Completo

```
public class CastingDemo {  
    public static void main(String[] args) {  
        double prezzo = 12.75;  
        int intero = (int) prezzo;  
        System.out.println(intero);  
        int eta = 20;  
        double etaDouble = eta;  
        System.out.println(etaDouble);  
    }  
}
```

Errori Comuni

Dimenticare il Cast

Errato:

```
double d = 3.14;  
int i = d;
```

Errore di compilazione.

Aspettarsi un Arrotondamento

```
double d = 9.99;  
int i = (int) d;
```

Risultato:

9

Non:

10

Perdita di Informazione

```
long valore = 10000000;  
short s = (short) valore;
```

Il valore risultante può essere diverso da quello originale.

Domande di Verifica

1. Cos'è il casting?
2. Qual è la differenza tra promotion e casting?
3. Quando il cast è obbligatorio?
4. Cosa accade convertendo un `double` in un `int` ?
5. Perché alcune conversioni possono provocare perdita di dati?
6. Cosa succede a `byte` , `short` e `char` nelle espressioni aritmetiche?
7. È possibile convertire un `boolean` in un `int` ?

Esercizi

Esercizio 1

Convertire un valore `double` in `int` e osservare il risultato.

Esercizio 2

Convertire un valore `int` in `double` .

Esercizio 3

Convertire un codice ASCII in carattere utilizzando `char` .

Esercizio 4

Convertire un carattere nel corrispondente valore numerico.

Esercizio 5

Verificare cosa accade convertendo numeri molto grandi in tipi più piccoli.

Esercizio 6

Leggere un numero da tastiera come stringa e convertirlo in intero.

Esercizio 7

Calcolare la media di due interi evitando errori dovuti alle conversioni implicite.

Metodi

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere il concetto di metodo;
- dichiarare e invocare metodi;
- utilizzare parametri e valori di ritorno;
- distinguere tra metodi `void` e metodi che restituiscono un valore;
- suddividere un problema in sottoproblemi mediante metodi;
- utilizzare l'overloading.

Introduzione

Un programma complesso può essere suddiviso in parti più piccole e più semplici da comprendere.

In Java queste parti prendono il nome di metodi.

Un metodo è un insieme di istruzioni identificato da un nome e progettato per svolgere un compito specifico.

I metodi permettono di:

- organizzare meglio il codice;
- evitare duplicazioni;
- migliorare la leggibilità;
- favorire il riutilizzo.

Cos'è un Metodo

Un metodo può essere visto come una scatola che:

- riceve dati in ingresso (parametri);
- esegue una serie di operazioni;
- restituisce eventualmente un risultato.

Esempio:

```
somma(3, 5)
```

Input:

```
3, 5
```


Output:

8

Struttura di un Metodo

Sintassi generale:

```
modificatore tipoRitorno nomeMetodo(parametri) {  
    istruzioni  
}
```

Esempio:

```
public static int somma(int a, int b) {  
    return a + b;  
    ---  
}
```

Componenti di un Metodo

Un metodo è composto da:

Elemento	Descrizione
Modificatore	ad esempio <code>public</code>
Tipo di ritorno	valore restituito
Nome	identificatore del metodo
Parametri	dati in ingresso
Corpo	istruzioni eseguite

Invocazione di un Metodo

Per eseguire un metodo è necessario invocarlo.

Esempio:

```
int risultato = somma(10, 20);
```

Il metodo viene eseguito e il valore restituito viene assegnato alla variabile.

Metodi Senza Parametri

Non tutti i metodi richiedono dati in ingresso.

Esempio:

```
public static void saluta() {  
    System.out.println("Ciao");  
}
```

Invocazione:

```
saluta();
```


Parametri

I parametri permettono di fornire dati al metodo.

Esempio:

```
public static void stampaNome(String nome) {  
    System.out.println(nome);  
}
```

Invocazione:

```
stampaNome("Mario");
```

Parametri Formali e Attuali

Nella dichiarazione:

```
public static int somma(int a, int b)
```

i parametri:

```
a  
b
```

sono parametri formali.

Nell'invocazione:

```
somma(3, 5);
```

i valori:

Metodi con Valore di Ritorno

Un metodo può restituire un risultato mediante l'istruzione `return` .

Esempio:

```
public static int quadrato(int numero) {  
    return numero * numero;  
}
```

Invocazione:

```
int risultato = quadrato(5);
```

Metodi void

Quando non è necessario restituire un valore si utilizza il tipo `void` .

Esempio:

```
public static void stampaMessaggio() {  
    System.out.println("Hello Java");  
}
```

L'Istruzione return

L'istruzione `return` termina l'esecuzione del metodo.

Esempio:

```
public static int massimo(int a, int b) {  
    if(a > b) {  
        return a;  
    }  
    return b;  
}
```

Variabili Locali

Le variabili dichiarate all'interno di un metodo esistono solo durante la sua esecuzione.

Esempio:

```
public static void esempio() {  
    int x = 10;  
}
```

La variabile `x` non è accessibile al di fuori del metodo.

Suddividere un Problema

I metodi consentono di decomporre un problema complesso.

Esempio:

```
public static void main(String[] args) {  
    leggiDati();  
    elaboraData();  
    stampaRisultati();  
}
```

Ogni metodo si occupa di un singolo compito.

Overloading

Java consente di definire più metodi con lo stesso nome purché abbiano parametri differenti.

Esempio:

```
public static int somma(int a, int b) {  
    return a + b;  
}  
  
public static double somma(double a, double b) {  
    return a + b;  
}
```

Il compilatore sceglie automaticamente il metodo corretto.

Esempio Completo

```
public class Calcolatrice {  
    public static int somma(int a, int b) {  
        return a + b;  
    }  
    public static int prodotto(int a, int b) {  
        return a * b;  
    }  
    public static void main(String[] args) {  
        int s = somma(10, 20);  
        int p = prodotto(5, 4);  
        ---  
        System.out.println(s);  
        System.out.println(p);  
    }  
}
```

Passaggio dei Parametri

In Java i parametri vengono passati per valore.

Esempio:

```
public static void incrementa(int x) {  
    x++;  
}  
  
public static void main(String[] args) {  
    int numero = 10;  
    incrementa(numero);  
    System.out.println(numero);  
    ---  
}
```

Output:

10

Metodi Predicativi

Un metodo predicativo restituisce un valore booleano.

Esempio:

```
public static boolean isPari(int numero) {  
    return numero % 2 == 0;  
}
```

Utilizzo:

```
if(isPari(10)) {  
    ---  
    System.out.println("Pari");  
}
```

Varargs

I varargs permettono di passare un numero variabile di argomenti.

Esempio:

```
public static int somma(int... valori) {  
    int totale = 0;  
    for(int v : valori) {  
        totale += v;  
    }  
    return totale;  
}
```

Utilizzo:

```
somma(1, 2);  
somma(1, 2, 3, 4);  
somma(10, 20, 30, 40, 50);
```


Errori Comuni

Dimenticare il return

Errato:

```
public static int somma(int a, int b) {  
    int risultato = a + b;  
}
```

Errore di compilazione.

Tipo di ritorno incompatibile

Errato:

```
public static int esempio() {  
    return "ciao";  
}
```

Errore di compilazione.

Numero errato di parametri

Metodo:

```
somma(int a, int b)
```

Invocazione errata:

```
somma(10);
```

Errore di compilazione.

Buone Pratiche

- utilizzare nomi significativi;
- realizzare metodi brevi;
- assegnare una singola responsabilità a ogni metodo;
- evitare duplicazioni di codice;
- documentare i metodi più complessi.

Domande di Verifica

1. Cos'è un metodo?
2. Qual è la differenza tra parametro formale e parametro attuale?
3. Quando si utilizza `void` ?
4. A cosa serve `return` ?
5. Cos'è l'overloading?
6. Cosa significa passaggio per valore?
7. Quando conviene utilizzare i varargs?

Esercizi

Esercizio 1

Scrivere un metodo che restituisca il quadrato di un numero.

Esercizio 2

Scrivere un metodo che restituisca il massimo tra due numeri.

Esercizio 3

Scrivere un metodo predicativo che verifichi se un numero è pari.

Esercizio 4

Scrivere un metodo che calcoli l'area di un rettangolo.

Esercizio 5

Scrivere un metodo che restituisca la lunghezza di una stringa senza utilizzare `length()`.

Esercizio 6

Scrivere un metodo che conti le vocali presenti in una stringa.

Esercizio 7

Scrivere un metodo con varargs che calcoli la somma di un insieme di numeri.

Esercizio 8

Scomporre un programma di gestione studenti nei metodi:

- inserisciStudente()
- calcolaMedia()
- stampaRisultati()

Esercizio 9

Realizzare una mini-calcolatrice composta esclusivamente da metodi.

Esercizio 10

Creare una libreria di metodi matematici riutilizzabili.

Numeri e Wrapper

Obiettivi

Al termine di questo capitolo sarai in grado di:

- comprendere la differenza tra tipi primitivi e classi wrapper;
- utilizzare le principali classi wrapper;
- convertire stringhe in numeri e viceversa;
- confrontare numeri rappresentati da oggetti;
- conoscere le problematiche legate alla precisione dei numeri decimali.

Introduzione

Nei capitoli precedenti abbiamo utilizzato i tipi primitivi:

```
int  
double  
boolean  
char
```

Java mette però a disposizione anche delle classi che rappresentano gli stessi valori sotto forma di oggetti.

Queste classi prendono il nome di wrapper.

Tipi Primitivi e Wrapper

Ogni tipo primitivo possiede una corrispondente classe wrapper.

Tipo primitivo	Classe Wrapper
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Perché Esistono i Wrapper

Le classi wrapper permettono di:

- trattare i valori primitivi come oggetti;
- utilizzare metodi di utilità;
- convertire stringhe in numeri;
- utilizzare collezioni e API avanzate.

Esempio:

```
Integer numero = 10;
```

Autoboxing

Java può convertire automaticamente un valore primitivo nel relativo wrapper.

Esempio:

```
int valore = 10;  
Integer oggetto = valore;
```

Questa conversione automatica prende il nome di autoboxing.

Unboxing

L'operazione inversa viene chiamata unboxing.

```
Integer oggetto = 10;  
int valore = oggetto;
```

Conversione di Stringhe in Numeri

Uno degli utilizzi più comuni dei wrapper consiste nella conversione di stringhe.

Integer

```
String testo = "123";  
  
int numero = Integer.parseInt(testo);
```

Double

```
String testo = "12.50";  
double prezzo = Double.parseDouble(testo);
```

Boolean

```
boolean attivo =  
    Boolean.parseBoolean("true");
```

Conversione di Numeri in Stringhe

```
int numero = 123;  
  
String testo = String.valueOf(numero);
```

Costanti Utili

Le classi wrapper forniscono costanti informative.

Esempio:

```
System.out.println(Integer.MAX_VALUE);  
System.out.println(Integer.MIN_VALUE);
```

Output:

```
2147483647  
-2147483648
```


Metodi Utili

Verifica di una Stringa Numerica

```
String valore = "123";  
  
int numero = Integer.parseInt(valore);
```

Conversione in Binario

```
System.out.println(  
    Integer.toString(10)  
);
```

Output:

```
1010
```

Conversione in Esadecimale

```
System.out.println(  
    Integer.toHexString(255)  
);
```

Output:

```
ff
```

Precisione dei Numeri Decimali

I tipi `float` e `double` utilizzano la rappresentazione floating point.

Questo può produrre risultati inattesi.

Esempio:

```
System.out.println(0.1 + 0.2);
```

Output:

```
0.30000000000000004
```

Introduzione a BigDecimal

Quando è richiesta precisione assoluta (ad esempio nelle applicazioni finanziarie) si utilizza la classe `BigDecimal`.

Esempio:

```
BigDecimal prezzo =  
    new BigDecimal("10.50");
```

L'argomento verrà approfondito nei capitoli dedicati alle API Java.

Formattazione dei Numeri

Java mette a disposizione diversi strumenti per formattare i numeri.

Esempio:

```
double prezzo = 1234.5;  
  
System.out.printf("%.2f%n", prezzo);
```

Output:

```
1234.50
```

Esempio Completo

```
public class DemoWrapper {  
    public static void main(String[] args) {  
        String testo = "25";  
        int eta = Integer.parseInt(testo);  
        Integer valore = eta;  
        System.out.println(eta);  
        System.out.println(valore);  
    }  
}
```

Errori Comuni

Conversione di una Stringa non Numerica

```
Integer.parseInt("abc");
```

Genera:

```
NumberFormatException
```

Confronto tra Wrapper con ==

Errato:

```
Integer a = 1000;  
Integer b = 1000;  
  
System.out.println(a == b);
```

Utilizzare:

```
a.equals(b)
```

Domande di Verifica

1. Cos'è una classe wrapper?
2. Qual è il wrapper associato a `int` ?
3. Cosa si intende per autoboxing?
4. Cosa si intende per unboxing?
5. A cosa serve `Integer.parseInt()` ?
6. Perché `double` può produrre risultati imprecisi?
7. Quando si utilizza `BigDecimal` ?

Esercizi

Esercizio 1

Convertire una stringa contenente un numero in un valore `int`.

Esercizio 2

Convertire una stringa contenente un prezzo in un valore `double` .

Esercizio 3

Stampare i valori massimo e minimo dei tipi `Integer`, `Long` e `Double`.

Esercizio 4

Convertire un numero decimale in rappresentazione binaria ed esadecimale.

Esercizio 5

Realizzare una semplice calcolatrice che acquisisca due numeri come stringhe e li converta in valori numerici.